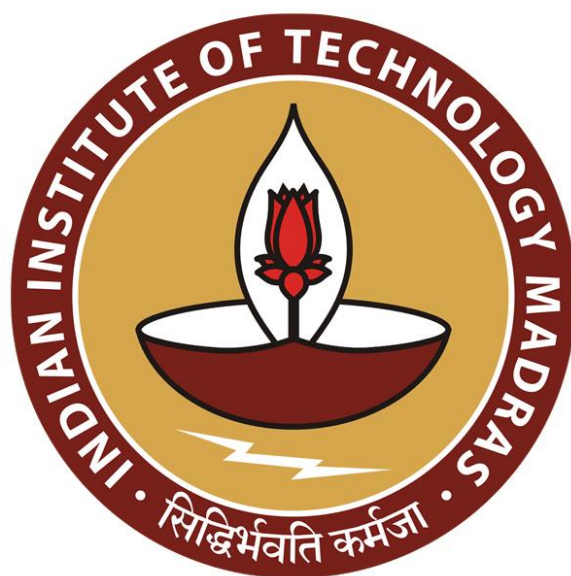




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Indian Institute of Technology, Madras
The Benefits of Indirection

One of the advantages of working with Java is that there is a vast library of built in data structures which are at your disposal. But before we look at these data structures in detail, let us look at a more abstract underlying principle as to how they are organized. So, this is to do with indirection. So, what do we mean by indirection?

(Refer Slide Time: 0:35)



Abstract data types

- Separate public interface from private implementation
- For instance, a (generic) queue

```
public class Queue<E> {  
    public void add (E element){...};  
    public E remove(){...};  
    public int size(){...};  
    ...  
}
```



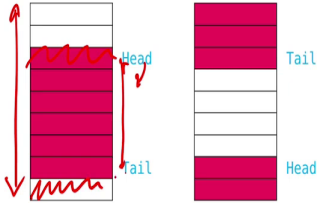


Madhavan MukundThe benefits of indirectionProgramming Concepts using Java 2 / 6



Abstract data types

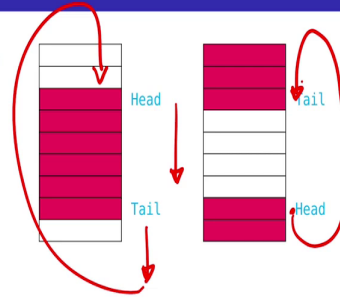
- Separate public interface from private implementation
- For instance, a (generic) queue
- Concrete implementation could be a circular array





Madhavan MukundThe benefits of indirectionProgramming Concepts using Java 2 / 6

- Separate public interface from private implementation
- For instance, a (generic) *queue*
- Concrete implementation could be a circular array



Well, let us first go back to the origin of abstract data types. So, one of the guiding principles of defining abstract data types was to separate the public interface from the private implementation. So the idea was that, if we had, for instance, something like this, which is a generic queue, then it has these public methods associated with namely, you can add an element at the end of the queue, you can remove an element from the front of the queue.

And maybe you can query the length of the queue by calling the size function. Now, when we come to a concrete implementation, this will have based on the implementation some way of reporting those things. So, here is something which is called a circular array. So, in a circular array, we have as you can see, an array. An array is just a block of storage.

And now, we designate where the queue starts and where the queue ends. So, here, the queue starts at this point, and it ends at this point, so the queue is actually between these two. Now, the idea is that if I remove an element from the queue, then this element will be deleted, and the head will move down by 1, if I add an element to the queue, then this element will get populated, and I will move the tail down by 1.

So, that is how the circular array works under normal circumstances. So, what will happen over time is that the head will move down and the tail will go beyond the boundary. And then, this is where the circularity comes in, it wraps around. So, here is a situation where the queue starts at the bottom, and actually continues to the top and comes here. So, this is a circular array implementation of a queue.

So, as we move the head down, we walk down the array and then as the tail moves down, if it reaches the end, it goes back to the top and comes from the top. So, this is one concrete way of implementing that. Now with respect to this, of course, I have already indicated how you would do the add and the remove.

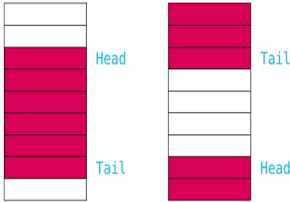
The removal, just take the element currently pointed to by head and then increment head. Tail would move the tail position down by 1 and populate the new value there, that would be the add operation and size will tell you the number of elements between head and tail modulo the circular shift.

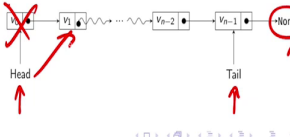
(Refer Slide Time: 2:41)



Abstract data types

- Separate public interface from private implementation
- For instance, a (generic) **queue**
- Concrete implementation could be a circular array
- Or a linked list





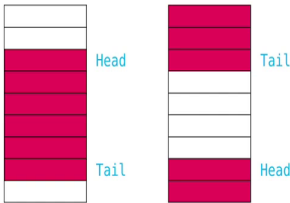


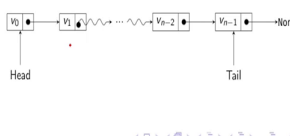
Madhavan Mukund The benefits of indirection Programming Concepts using Java 2 / 6



Abstract data types

- Separate public interface from private implementation
- For instance, a (generic) **queue**
- Concrete implementation could be a circular array
- Or a linked list
- Implementer of class **Queue** can choose either one
- Public interface is unchanged







Madhavan Mukund The benefits of indirection Programming Concepts using Java 2 / 6

But the question is, although the functionality that is the way that these functions behave is not affected by the implementation, is it true that the user is actually indifferent to these choices? So, we can see that these choices are very different in the way they represent the data and this has some implications for performance, for example.

So, the interface only talks about the input output behavior of the function, it says, if I give you if I call a function, then this is what I expected, the current state of the data structure is this and if I call a function, and this is the value that I expect to get back. It does not tell you how long it will take or how efficient it is going to be.

So, if you look at efficiency, for instance, it appears that this is better. And the reason for that is that we have effectively one block of storage which we start off with and after that we are just working within this block. So, there is no extra work as the queue shrinks, the queue grows and shrinks, the storage remains the same way, just manipulating a pointer this head and tail, these indices within the existing storage.

So, accessing the queue, and updating the queue is a very efficient thing. However, notice that we are limited by the number of elements that we allocated to start with. So, if we want a flexible queue that can grow to any size, then the circular array is likely to run into problems because we run out of space in case the queue becomes very large. So, in such a situation, the linked list might be better.

So, if we have no clear indication at the beginning of how large our queue might be, then we may prefer a linked list. So, therefore these two are not quite the same, even though from a interface perspective, they are identical, there are observable differences from the point of view of usability. So, to get around this, we might tell the user that there are two different implementations and you -choose.

(Refer Slide Time: 5:14)

Multiple implementations

- Create two separate implementations

```
public class CircularArrayQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```

```
public class LinkedListQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```



Madhavan Mukund

The benefits of indirection

Programming Concepts using Java 4/6

So, we can probably create two separate classes, instead of having that one class queue, which had a hidden implementation which the user had no control over. Now we provide the user with two implementations. So, if you want a circular array queue, you pick up this one, if you want a linked list, you pick up this one. Now the interface is the same, so these three functions are the same. So, as far as using it, there is no problem. But it's a question of the other aspects, efficiency and flexibility, and so on.

(Refer Slide Time: 5:42)

Multiple implementations

- Create two separate implementations

- User chooses

```
CircularArrayQueue<Date> dateq;
LinkedListQueue<String> stringq;
```

```
dateq =
    new CircularArrayQueue<Date>();
stringq =
    new LinkedListQueue<String>();
}
```

- What if we later realize we need a flexible size `dateq`?

- Change declaration for `dateq`

```
public class CircularArrayQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```

```
public class LinkedListQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```



Madhavan Mukund

The benefits of indirection

Programming Concepts using Java 4/6

- Create two separate implementations
- User chooses


```
CircularArrayQueue<Date> dateq;
LinkedListQueue<String> stringq;
```

```
dateq =
    new CircularArrayQueue<Date>();
stringq =
    new LinkedListQueue<String>();
```
- What if we later realize we need a flexible size `dateq`?
- Change declaration for `dateq`
- And also every function header, auxiliary variable, ... associated with it

```
public class CircularArrayQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}

public class LinkedListQueue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```



Madhavan Mukund

The benefits of indirection

Programming Concepts using Java 4 / 6

So, the user might choose, for instance, to have a queue of dates using a circular array queue (`CircularArrayQueue<Date> dateq;`), and a queue of strings using a list a linked list queue (`LinkedListQueue<String> stringq;`). And so when you create it, of course, you then define it, you declare it up here, and then you instantiate it. Similarly, you declare it up here and you instantiate it.

So, this is one solution to this problem of providing different implementations with different features and then we allow the user to choose the implementation that they prefer. So, what is the downside of this? Well, the downside, or the disadvantage of this is that now, we might not understand when we start running our code, whether one implementation is preferable to the other one. So, for instance, in this case, we chose the circular array queue for dates.

And later on, we might discover that due to some unforeseen way in which our application is being used, that is not really a practical assumption to assume that the array size is fixed in advance, we actually need a flexible one. So, how do we fix this? Well, we have to go back in our code, and we have to change this declaration. And of course, we have to change the corresponding call here.

But this is not all, we would have taken this data structure, and we would have been using it all over the place in our code. So, wherever we use it, wherever we have a function that uses this `dateq`, we would have used the circular array queue as an argument. And wherever we declared some new temporary variables to hold, say, a parallel queue, while we are doing something with it, we would have again, use a circular array queue, well.

There will be a lot of instances of this circular array queue, which have crept into our code as a result of this design decision at the beginning, that we are going to use a circular array queue for dateq. And now if I want to change that particular design decision, to a linked list queue, I have to be careful to go and fix all these places where I have made this change. So, how do we get around this particular disadvantage?

(Refer Slide Time: 7:42)



Adding indirection

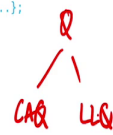
- Instead, create a `Queue` interface
- Concrete implementations implement the interface

```

public interface Queue<E> {
    abstract void add (E element);
    abstract E remove();
    abstract int size();
}

public class CircularArrayQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}

public class LinkedListQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
        
```





Madhavan Mukund
The benefits of indirection
Programming Concepts using Java
5 / 6



Adding indirection

- Instead, create a `Queue` interface
- Concrete implementations implement the interface
- Use the `interface` to declare variables

```

Queue<Date> dateq;
Queue<String> stringq;

dateq =
    new CircularArrayQueue<Date>();
stringq =
    new LinkedListQueue<String>();
        
```

```

public interface Queue<E> {
    abstract void add (E element);
    abstract E remove();
    abstract int size();
}

public class CircularArrayQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}

public class LinkedListQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
        
```



Madhavan Mukund
The benefits of indirection
Programming Concepts using Java
5 / 6

- Instead, create a `Queue` interface
- Concrete implementations implement the interface
- Use the **interface** to declare variables
`Queue<Date> dateq;`
`Queue<String> stringq;`

`dateq =`
`new CircularArrayQueue<Date>();`
`stringq =`
`new LinkedListQueue<String>();`
`}`
- Benefit of **indirection** — to use a different implementation for `dateq`, only need to update the instantiation

```
public interface Queue<E> {
    abstract void add (E element);
    abstract E remove();
    abstract int size();
}

public class CircularArrayQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}

public class LinkedListQueue<E>
    implements Queue<E> {
    public void add (E element){...};
    public E remove(){...};
    public int size(){...};
    ...
}
```



Madhavan Mukund

The benefits of indirection

Programming Concepts using Java 5/6

So now, we come back to our favourite interfaces. So, Java provides us with this interface, which is capturing purely the capabilities. So, if we create, our first thing was to create an abstract class queue. So, we created this queue, which had a public interface, and we said, the private implementations. So, what if we keep only the interface part? So, instead of making it a class make it an interface.

So, we just provide these three abstract functions, add, remove and size, and say that now a real queue will actually implement this. Now, what we do is we make our two concrete implementations, the ones that we just described before, instead of making them separate implementations where the user has to choose, we are there still separate implementations that the user has to choose from, but they both implement this interface. So, they are both tied to this.

So, we have this queue on the top, which is an interface and below that these are both kind of children of that class in terms of the class hierarchies, so the circular array queue and the linked list queue are both now implementations of that. So, since they are both implementations of that now, we can use this compatibility of these class names across this hierarchy. And now we can use the interface name to declare.

So, instead of committing that a `dateq` is going to be a circular array queue and a `stringq` is going to be a linked list queue, we just say that they are both queues. And this is enough given that the interface is implemented, it is enough to know that the public functions available for queue are available here.

And thanks to the fact that we have this dynamic dispatch, the actual choice of these functions will be guided by the actual implementation. So, when I create the concrete queue, `dateq` is expecting an arbitrary queue, but it is willing to take a circular array queue. So, now if I take the `dateq` and I do `add remove`, it will apply those functions as they are defined in circular array queue.

And similarly, `stringq` will apply the functions as they are applied in linked list queue. So, we do not seem to have done much. We basically have postponed this decision of where to choose between the two implementations to the instantiation. So, the main thing is that we are not declaring it upfront to be of one concrete type or the other concrete type, we are using this abstract interface. And now we are postponing the instantiation, or the choice to the instantiation.

But the advantage of this is that this is called indirection, I am not referring to a concrete, this queue or that queue type, I am saying it is a queue. So, I am kind of pointing to an abstract concept, which represents the concrete thing that I want. So, I am indirectly saying that I am choosing one of these two, but I will tell you later on which one I am choosing. So, this is this indirection.

So, the advantage of indirection is supposing I change my decision, I have to of course, go back and re instantiate `dateq` correctly. But presumably, since `dateq` itself was defined only to be an abstract queue, every function and every other auxiliary variable that I wrote throughout my code, was also just declared to be an abstract view. So, all those declarations do not have to change anymore.

So, the amount of update that I have to make to my code is limited. So, the benefit of using this interface to capture the abstract part, and then separate out the concrete part into an implementation of the interface gets us a lot of benefits in terms of updating the code if we happen to choose the implementation later.

(Refer Slide Time: 11:18)

- Use interfaces to flexibly choose between multiple concrete implementations
- Interfaces add a level of *indirection*
- Indirection in real life
 - Organization provides senior staff with an office car
 - Concrete: each official has an assigned car — what if it breaks down?
 - Indirection: a pool of office cars, use any that is available
 - Don't want to maintain a pool of cars? Contract with a taxi service
 - Don't want to negotiate tenders? Reimburse taxi bills

Xenos P.H.C.



"Fundamental theorem of software engineering"

All problems in computer science can be solved by another level of indirection.

Butler Lampson, Turing Award 1992

Madhavan Mukund

The benefits of indirection

Programming Concepts using Java

6 / 6

So, this is the kind of way in which Java organizes data structures. So, it is important to understand this motivation for it. So, basically, we can use interfaces to flexibly choose between multiple concrete implementations. So, that is the real moral of the story. So, interfaces, in addition to all the other things.

So, remember, we first introduced interfaces partly, it looked like a reaction to the fact that Java does not support multiple inheritance. So, we said if you want to keep track of multiple capabilities in a class, we allowed it to support these multiple abstract interfaces. But interfaces actually have a much larger role to play in some sense in the context of object oriented programming and object oriented thinking, they give you this level of indirection.

So, just to drive home, this idea of indirection, so that you know, you have something to that tangible that you can remember it by. So, suppose we have an organization in which senior staff are provided with an office car, it is not unusual that people who are at a senior level in an organization are provided with transportation by the company.

So, if I take this literally, if I take the concrete implementation view of this, then each official who has access to an office car will be assigned a car. So, the organization will maintain a fleet of cars, and one car will be assigned to one employee who is entitled to that car. So, they will be for example, the President's car, there will be the Vice President's car, the directors car, and so on.

So, now the problem with this concrete assignment is that each car, of course, is prone to its own problems. And so if a car breaks down, then that particular official is deprived of a car

till the problem is resolved. So, how do we make this indirect? Well, we can say that an office car is not a concrete object, but it is an abstract thing, it is a transportation device, which is provided by the office to this official for office use.

Now, it is no longer required that this particular vehicle must be maintained as an individual vehicle for that particular officer. So, for in particular, for instance, we could imagine that the office itself does not allocate one car per staff member, and maybe it has a few spare cars. So, maybe if there are 5 people who are entitled to cars, the office actually keeps maybe 6 or 7 cars in its pool. And at any given time, when an official needs a car, one of the cars from this pool is assigned.

So this means that even if there is a probability of 1 or 2 cars being under maintenance, not every official will need their office car at the same time, presumably, but also there will be a possibility of taking an alternative car in case the car that they normally prefer is not available. So, this is one possible way in which indirection will work.

But of course, the benefit of indirection is that this is not now the only solution to the problem, our problem was we wanted to provide an office car, but we did not want to fix the office car because that has problems of this flexibility, because the office car breaks down we cannot do it. So, one possibility is to have a pool of cars and not assigned.

But we do not even have to maintain a pool of cars. Supposing we do not want to have this extra headache of maintaining a workshop or people train mechanics to deal with these cars, we can convert this whole thing into what is now in economics called an outsourced model. So, we could now have a contract with a taxi company and say that whenever an office officer needs a car, we will call through you and you must provide it.

So, now this means that the cars themselves and this whole headache of men ensuring that a car is available and in good condition and all is transferred to the taxi company. But as far as the state, this organization is concerned and the officer who is entitled to a car is concerned, he or she has an office car at their disposal. So, it is still a solution to the original abstract question through indirection.

And one could go even one step further, supposing you do not want to go through this hassle of negotiating a contract with a taxi company, because then you have to go through tendering and all kinds of other processes that your organization might require, you might just strike a

deal with these people who are entitled to office car saying you take a taxi and provide a bill and we will reimburse it.

So, effectively now that the burden of arranging the office car has been transferred back to the official but still the cost involved with this is no longer there. So, this is an example of how indirection can make the solution to a problem extremely flexible compared to a rigid solution, where you have to choose one implementation or another implementation.

So, in fact, there is a kind of half humorous saying attributed to this very well-known computer scientist called Butler Lampson who won the Turing Award in 1992. And Butler Lampson was one of the original researchers at this organization called Xerox PARC. And he is widely credited with everything that we are now used to as a personal computer.

In the days when all computers were these large wardrobe size things which fitted in one big room, he had the vision to imagine a personal computer with a mouse and kind of keyboard interface which people could use and so on. So, anyway, so what Butler Lampson said is that any problem all problems in computer science can be solved by adding one more level of indirection. So, do not fix, do not insist that this must be give it a name.

And then later on, decide what you will do to fill that name. So, in fact, this has jokingly be called the Fundamental Theorem of Software Engineering. So, it is, in a way it is a bit of postponement, you are postponing the decision to as late as possible, so that you are not stuck with an early decision, which is suboptimal.

So, this is the principle, as I said, behind which this Java collections, as they are called are organized. So, it is useful to understand why they are doing this, because otherwise it might look a little strange, that something as simple as providing, a bunch of user system defined data structures should be so bureaucratically complicated.